

Joint Service Migration and Resource Allocation in Edge IoT System Based on Deep Reinforcement Learning

Fangzheng Liu[✉], Hao Yu[✉], *Member, IEEE*, Jiwei Huang[✉], *Senior Member, IEEE*,
and Tarik Taleb[✉], *Senior Member, IEEE*

Abstract—Multiaccess edge computing (MEC) provides services for resource-sensitive and delay-sensitive Internet of Things (IoT) applications by extending the capabilities of cloud computing to the edge of the networks. However, the high mobility of IoT devices (e.g., vehicles) and the limited resources of edge servers (ESs) affect the service continuity and access latency. Service migration and reasonable resource (re-)allocation consequently become needed to ensure Quality of Service (QoS). However, service migration results in additional latency. In addition, different mobile IoT users have different resource requirements and different resource allocation policies of target ESs also determine whether service migration is necessary. Subsequently, how to jointly optimize service migration and resource allocation (SMRA) is a challenge that needs to be carefully addressed. To this end, this article investigates the joint optimization problem of SMRA in MEC environments to minimize the access delay of IoT users. It proposes a joint SMRA algorithm based on deep reinforcement learning (DRL), which takes into account the mobility of IoT users and decides whether to migrate services, where to migrate, and how to allocate resources through the long short time memory (LSTM) algorithm and the parameterized deep Q -network (PDQN) algorithm. Moreover, the PDQN algorithm effectively solves the discrete-continuous hybrid action space challenge in the SMRA problem. Finally, we conduct evaluation using a real-world data set of Beijing cab trajectories to verify the effectiveness and superiority of our proposed SMRA solution.

Index Terms—Cloud, deep reinforcement learning (DRL), edge computing, Internet of Things (IoT), long short term memory (LSTM), multiaccess edge computing (MEC), parameterized deep Q -networks (PDQNs), resource allocation, service migration.

Manuscript received 4 July 2023; revised 22 September 2023 and 10 October 2023; accepted 29 October 2023. Date of publication 14 November 2023; date of current version 26 March 2024. This work was supported in part by the National Natural Science Foundation of China under Grant 61972414; in part by the Beijing Nova Program under Grant Z201100006820082; in part by the European Union's Horizon Europe Program for Research and Innovation through the aerOS Project under Grant 101069732; in part by the Business Finland 6Bridge 6Core Project under Grant 8410/31/2022; in part by the Research Council of Finland 6Genesis Project under Grant 318927; and in part by the Research Council of Finland IDEA-MILL Project under Grant 352428. This work was also carried out in ICTFICIAL OY. (Corresponding author: Jiwei Huang.)

Fangzheng Liu and Jiwei Huang are with the Beijing Key Laboratory of Petroleum Data Mining, China University of Petroleum, Beijing 102249, China (e-mail: 2019310704@student.cup.edu.cn; huangjiw@cup.edu.cn).

Hao Yu and Tarik Taleb are with the Center of Wireless Communications, University of Oulu, 90570 Oulu, Finland (e-mail: Hao.Yu@oulu.fi; tarik.taleb@oulu.fi).

Digital Object Identifier 10.1109/JIOT.2023.3332421

I. INTRODUCTION

MULTIACCESS edge computing (MEC) is a new computing paradigm which solves the access delay problem of traditional cloud computing by deploying services at edge servers (ESs) closer to Internet of Things (IoT) users. As IoT devices are becoming increasingly intelligent, emerging IoT applications (i.e., autonomous driving, online gaming, ultrahigh-definition video, etc.) place high requirements on service performance and resources. Nevertheless, ESs are normally equipped with limited heterogeneous resources and have limited coverage, while different IoT users have different resource requirements and different mobility patterns. For delay-sensitive and resource-intensive IoT applications, how to improve resource utilization and reduce access delay while maintaining service continuity is a huge challenge for edge system performance optimization.

As shown in Fig. 1, we consider the following scenario whereby user u_1 connects to ES e_1 at time t and requests service s_2 on ES e_1 . We assume that user u_1 will move to the vicinity of ES e_3 at time $t + 1$. On the one hand, if the user continues to connect to the source ES e_1 , it will result in a long communication delay between the mobile user u_1 and the source ES e_1 . On the other hand, we can perform service migration to migrate the service s_2 from the source ES e_1 to the target ES e_3 , so as to shorten the communication distance and reduce the communication delay. Service migration is an effective means of ensuring service continuity, but it will additionally induce the migration delay. In addition, performing service migration will change the association between IoT users and ESs, and we need to consider the heterogeneity and limited resources of the target ESs. The resource allocation policy of target ESs determines the computation latency and communication latency of mobile users after service migration, which in turn affects the decision of whether to migrate or not. Therefore, we need to consider the service migration problem and resource allocation problem jointly and only perform service migration that is necessary and beneficial for reducing access delay.

To ensure service continuity and reduce access delay, future smart IoT systems should be able to perceive IoT users' mobile behavior and have the ability to execute service migration and resource allocation (SMRA) schemes in advance.

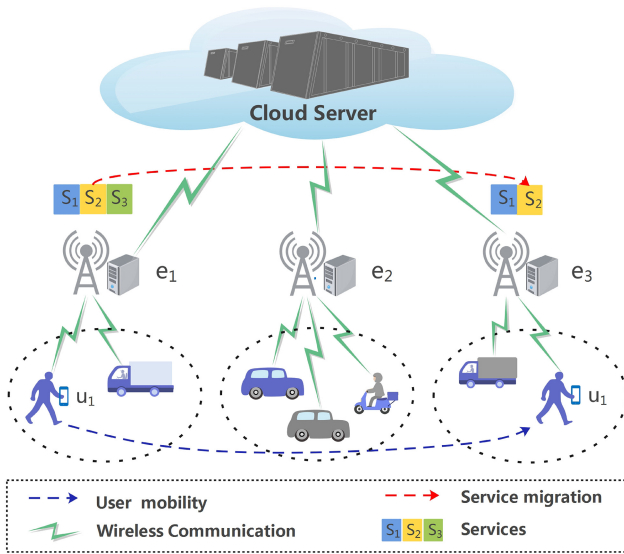


Fig. 1. Service migration scenario.

Unfortunately, most previous research works have studied SMRA as separate optimization problems, ignoring the fact that service migration decision and resource allocation scheme are mutually influential. Service migration is the factor that affects service continuity, while resource allocation strategy is another factor affecting the service migration decision. In addition, different IoT users may have different resource requirements and mobility patterns. Furthermore, the resources of ESs may be limited and heterogeneous. They dynamically change over time. These characteristics increase the difficulty of jointly optimizing SMRA. To fill the above gaps, we investigate the SMRA problem by jointly considering these two factors to determine whether service migration is needed, where to move the services, and how to allocate resources, ultimately, to accommodate the mobility of IoT users, optimize IoT system resource utilization, minimize user access delay, effectively avoid service disruptions, and improve overall Quality of Service (QoS).

With the above background, we try to solve the joint optimization problem of SMRA, establish a joint optimization model by integrating resource constraints, and propose a deep reinforcement learning (DRL)-based SMRA algorithm which is able to predict mobile behavior of users and find an optimal service migration policy and resource allocation scheme to ensure the service continuity and minimize the access delay of users. Specifically, the main contributions of this article are summarized as follows.

- 1) Different from existing studies, this article investigates the joint optimization problem of dynamic SMRA, comprehensively considering the heterogeneity and limited resources of ESs, as well as the different resource requirements and mobility of IoT users. The obtained optimal service migration policy and resource allocation scheme can ensure service continuity, minimize the access delay of users and improve resource utilization under the limited communication and computing resource constraints.

- 2) Considering the mobility of IoT users and the dynamic changes of available resources of ESs, this article formulates the joint optimization SMRA problem as a Markov decision process (MDP) and proposes a DRL-based SMRA algorithm, which uses long short term memory (LSTM) predicting the mobility behavior of IoT users to improve migration accuracy and reduce the action space. Then, the parameterized deep Q -network (PDQN) algorithm is used to solve the continuous-discrete hybrid action space challenge faced in the joint optimization process without destroying the original structure of the action space.
- 3) In order to verify the performance of our SMRA algorithm, we conducted experiments using the Beijing cab GPS track data set. The experimental results indicate that our SMRA algorithm can obtain a smaller average task processing latency and has better performance compared to other baseline methods.

The remainder of this article is organized as follows. In Section II, some related work are reviewed and the remaining challenges of SMRA are summarized. Section III presents the system model and the problem formulation. Section IV analyzes the SMRA problem and proposes a DRL-based SMRA algorithm. Section V evaluates the performance of the SMRA algorithm with real-world cab trajectory data set. Section VI concludes this article.

II. RELATED WORK

Currently, there are few studies on the joint optimization of SMRA problems, so we mainly discuss the related work from the aspects of SMRA, separately.

A. Service Migration

Service migration is an effective way to ensure service continuity and avoid service disruptions [1]. There are already some preliminary studies on service migration. Ouyang et al. [2] presented using the Lyapunov optimization technique to address the challenges of service placement and service migration, which focused on the case where an ES allocates an equal amount of computational resources to the users it serves. However, the fact is that resource requirements vary from user to user and change dynamically over time. Mada et al. [3] presented a comprehensive analysis of user mobility, service types, and system environment characteristics, and gave an initial placement and migration scheme for the service. This study initially solve the service migration problem but ignores the impact of the resource allocation policy of the target ES on the service migration results. Since different ESs have different available resources, if the target ES has fewer resources (e.g., computation resource and communication resource) relative to the source ES and cannot allocate enough resources to IoT users, it will inevitably increase the computation and communication time of tasks after service migration, which in turn affects the result of whether to migrate or not.

Chen et al. [4] proposed a dual-delay deep deterministic policy gradient (DDPG) algorithm to address long-term dynamic

task allocation and service migration issues. Nevertheless, the mobility of users is unknown, and if the system performs the service migration operation after the users have deviated from the coverage of the source ES will increase the access latency and degrade the user experience. Thus, it is necessary for the system to be able to predict the mobility of users and perform migration operations in advance so that seamless handover can be achieved and thus service continuity can be guaranteed. Wang et al. [5] proposed to solve the service disruption problem caused by user mobility through base station (BS) switching, however, the authors neglected the change in computational resources due to BS switching. In contrast, our proposed scheme considers user mobility and the change in resources during service migration. This complicated issue has not been studied in service migration.

In addition, there are several studies focusing on service slice migration and resource management. Addad et al. [6] designed, modeled, and evaluated two DRL-based algorithms for allowing a fine-grained selection of system-based triggers regarding network slice movement patterns. This work aims to make their defined triggers more intelligent while keeping system resources stable, but does not ensure reduced network resource overhead. To address this issue, Addad et al. [7] further developed a network-aware agent capable of selecting accurate bandwidth values while ensuring fast and reliable service migration. However, system resources include not only bandwidth resources but also computational resources, and we need to manage system resources in an integrated manner to achieve optimized QoS.

B. Resource Allocation

In the IoT environment, the requirements of mobile users for resources and the remaining resources of ESs are dynamically changing over time, and unreasonable resource allocation strategies will have an impact on service migration decisions. Liang et al. [8] proposed to solve SMRA by a relaxation and rounding-based approach with the aim of maximizing the weighted sum of the offload rate and migration cost. This article mainly considered a static and stable edge computing scenario, and the authors assumed that the migration cost of each user is fixed, ignoring the dynamic nature of system state. However, in real IoT systems, the available resources of the ESs and the location of IoT users change over time, and SMRA policies need to be dynamically adjusted according to the real-time changes of system to ensure service performance and resource utilization. In addition, the authors did not consider the real computing power (e.g., CPU cycles) of ES in order to simplify the model, but defined computing power as the number of users it can serve.

For multiuser MEC systems, dynamic service migration decisions and resource management are more complicated since it involves sharing of resources among multiple mobile users. However, existing studies on the dynamic optimization of resource allocation for multiple users mainly focus on the optimization of offloading policies. Liu et al. [9] considered stochastic vehicular traffic, dynamic computational requests and time-varying communication conditions, and proposed a

DRL-based approach to find computational offloading and resource allocation strategies in vehicular edge computing networks to maximize the long-term utility of the network. Dai et al. [10] focused on the changing communication and computational resources in an end-edge-cloud coordination network and proposed a DRL-based computational offloading and resource allocation algorithm to reduce energy consumption. To obtain discrete offloading decisions, the authors employed a rounding technique to refine the continuous values output by the DDPG algorithm, but this will inevitably affect the results. In contrast, we use the PDQN algorithm applicable to the continuous-discrete hybrid action space to solve the complex model of SMRA established in this article.

Different from the above studies, we explore an unknown direction, and investigate the joint optimization problem of dynamic SMRA in a multiuser IoT environment, considering the heterogeneity of ESs and limited resources, as well as the resource requirements and mobility of different IoT users, with the aim of ensuring service continuity, minimizing user access time, and maximizing resource utilization.

III. SYSTEM MODEL AND PROBLEM FORMULATION

In this section, we first illustrate the system model. We then formulate the joint optimization SMRA problem as a total latency minimization problem under communication and computational resource constraints. The key notations, used in this article, are summarized in Table I.

A. System Model

As shown in Fig. 1, we consider an IoT system which includes a set of mobile users $\mathcal{U} \triangleq \{1, 2, \dots, U\}$, a set of BSs integrating ESs $\mathcal{E} \triangleq \{1, \dots, E\}$ and a cloud server. We assume that in the case where a mobile user does not establish a connection with any other BS, it can only connect to the nearest BS and can only access the resources on the ESs equipped by that BS [10], [11], [12]. We consider a scenario where each mobile user has a latency-sensitive computational task to process. We use $u \in \mathcal{U}$ and $e \in \mathcal{E}$ to represent the u th mobile user and the e th ES, respectively. We use $D_{u,e}$ to denote the input data-size (in bits) for processing the task and $C_{u,e}$ represents the number of CPU cycles required to compute one-bit data of this task. Each BS is equipped with an ES, which has limited computational and transmission resources. In contrast, the cloud server have powerful computing resources and acts as a controller of the IoT system in this article which is responsible for SMRA.

On each ES, the underlying operating system (OS) is used as an underlying support environment to provide support for the task processing of different intelligent services and the operation of upper layer software. On the OS, various edge service functions (SFs) at the edge are deployed to meet the specialized task processing needs of different applications. At the same time, SFs store and manage stateless contextual information, such as public databases, executable code, etc., that is relevant only to a particular type of business but not to a particular user. Each SF can serve multiple users performing the same type of service. Once the user's computational task

TABLE I
LIST OF KEY NOTATIONS

Notation	Description
$r_{u,e}$	Data communication rate of user u to edge server e
$b_{u,e}$	Channel bandwidth
p_u	Transmission powers of user u
$g_{u,e}$	Channel gain between mobile user u and edge server e
σ^2	Gaussian noise power
$D_{u,e}(t)$	The task data size of user u
$I_{u,e}^{comm}(t)$	Transmission time of task data from user u to edge server e
$f_{u,e}(t)$	Computation resource allocated to user u by edge server e
$C_{u,e}(t)$	The number of CPU cycles for computing one-bit task data
$I_{u,e}^{comp}(t)$	Computation time for ES e to process the tasks of user u
$I_{u,e}^{sus}(t)$	SI suspending time of user u
$D_{u,e}^{mig}(t)$	The size of state context data of user u
$C_{u,e}^{sus}(t)$	The processing intensity required to suspend the SI of user u
$\beta_{u,e}(t)$	The proportion of computational resources allocated to checkpoint function
$f_{u,e}^{SF}(t)$	The computing capacity of SF required by user u
$I_{u,e}^{res}(t)$	SI resuming time of user u
$C_{u,e}^{res}(t)$	The processing intensity required to resume the SI of user u
$I_{u,e}^{syn}(t)$	State context data synchronization time
$r_{e,e'}(t)$	Data communication rate of edge server e to e'
$I_{u,e}^{mig}(t)$	The service migration time of user u
$I_{u,e'}^{comm}(t)$	Transmission time of task data from user u to edge server e'
$I_{u,e'}^{comp}(t)$	Computation time of ES e' to process tasks from user u
$f_{u,e'}(t)$	Computation resource allocated to user u by edge server e'
$I_{u,e'}^{nm}(t)$	Total task processing delay in non-migration case
$I_{u,e'}^m(t)$	Total task processing delay in migration case
$I_u(t)$	Total task processing delay of user u at time t
$w_{u,e,e'}^{nm}(t)$	Binary service non-migration decision variable
$w_{u,e,e'}^m(t)$	Binary service migration decision variable
$\alpha_{u,e}(t)$	Link relationship between user u and edge server e at time t
$F_{u,e'}(t)$	Available CPU capacity of edge server e' at time t
$r_{e,e'}(t)$	Data communication rate of edge server e to e'
$B_{u,e'}(t)$	Available bandwidth of edge server e' at time t
$L_{u,e'}^{pre}(t)$	The predicted location of user u at time $t+1$
$L_{u,e'}^{pre}$	Location of the nearest edge server to the predicted user location.

is assigned to an SF that has been activated on the ES for processing, the SF first instantiates a service instance (SI) for that user, which is dedicated to processing the user's computation task and storing and managing some user-specific state context data, such as task real-time processing status, private data, etc.

As in [13], a mobile user sends requests to the nearest ES by a wireless link connection. Therefore, based on the Shannon formula, the data communication rate (in bit/s) of mobile user u is given by

$$r_{u,e} = b_{u,e} \log_2 \left(1 + \frac{p_u g_{u,e}}{\sigma^2} \right) \quad (1)$$

where $b_{u,e}$ is the channel bandwidth, $g_{u,e}$ represents the channel gain, which is related to the distance between mobile user u and ES e , p_u denotes the transmission power of mobile user u , and σ^2 is the white Gaussian noise power.

In this article, we will optimize and update the IoT system according to each time slot t . At the beginning of each time slot, the IoT system controller needs to make a choice based on migration delay and nonmigration delay to determine whether to migrate or not, as well as the resource allocation policy for target ES in case of migration.

1) *Nonmigration Model*: If the service does not need to be migrated, the delay of mobile user u accessing the source ES e consists mainly of communication delay and computation delay.

1) *Communication Delay*: The wireless transmission delay of tasks offloading from mobile user u to source ES e can be written by

$$I_{u,e}^{comm}(t) = \frac{D_{u,e}(t)}{r_{u,e}(t)}. \quad (2)$$

Since the size of the result data obtained after task processing is very small relative to the size of task data, we omit the transmission delay of the downlink in this article.

2) *Computing Delay*: After receiving the tasks from mobile users, the source ES will allocate the available computing resources to each task for computation. We denote the computational resources allocated by the source ES e to the mobile user u by $f_{u,e}(t)$ (in CPU cycles/s). Thus, the computational delay of the source ES e to process tasks from mobile user u can be written as

$$I_{u,e}^{comp}(t) = \frac{D_{u,e}(t) C_{u,e}(t)}{f_{u,e}(t)}. \quad (3)$$

To sum up, when a mobile user moves from the source ES e to the vicinity of the target ES e' , if no service migration is performed and user u continues to access the source ES e , the task processing delay can be expressed as follows:

$$I_{u,e}^{nm}(t) = I_{u,e}^{comm}(t) + I_{u,e}^{comp}(t). \quad (4)$$

2) *Migration Model*: In the case where services need to be migrated, the total delay from the start of executing service migration until the mobile user u completes access to the target ES e' includes the following.

- 1) The service migration delay from the source ES e to the target ES e' .
- 2) The communication delay between the mobile user u and the target ES e' .
- 3) The computational delay of the target ES e' to perform the tasks from the mobile user u .

a) *Migration delay*: Service migration aims to synchronize user-dependent context data from source ES to target ES. As in [4], executing service migration includes the following steps: suspending the user's SI; preparing the user-dependent SF environment; synchronizing the user's state context data; and resuming the user's SI and service processes.

The suspend and resume of user's SIs can be implemented using Checkpoint functional modules [14]. In this article, we assume that this function module is embedded in each SF with constant processing power. Therefore, the SI suspend time $I_{u,e'}^{sus}(t)$ and resume time $I_{u,e'}^{res}(t)$ for user u at time t are, respectively, given by

$$I_{u,e'}^{sus}(t) = \frac{D_{u,e'}^{mig}(t) C_{u,e'}^{sus}(t)}{\beta_{u,e'}(t) f_{u,e'}^{SF}(t)} \quad (5)$$

$$I_{u,e'}^{res}(t) = \frac{D_{u,e'}^{mig}(t) C_{u,e'}^{res}(t)}{\beta_{u,e'}(t) f_{u,e'}^{SF}(t)} \quad (6)$$

where $D_{u,e'}^{mig}(t)$ (in bits) represents the size of the state context data of user u , $f_{u,e'}^{SF}(t)$ represents the computing capacity of the SF required by u (in CPU cycles/s), $\beta_{u,e'}(t)$ is the

proportion of computational resources allocated by the SF to the embedded Checkpoint function, and $C_{u,e}^{\text{sus}}(t)$ and $C_{u,e'}^{\text{res}}(t)$ (in CPU cycles/bit) represent the processing intensity required to suspend SI and resume SI of user u , respectively, (i.e., the number of CPU cycles required by compute one-bit of state context data).

The synchronization time of the user state context data relies on the state context data size and the data transmission rate between source ES and target ES [2], which is written as

$$I_{u,e'}^{\text{syn}}(t) = \frac{D_{u,e'}^{\text{mig}}(t)}{r_{e,e'}(t)} \quad (7)$$

where $r_{e,e'}$ denotes the data communication rate between the source ES e and the target ES e' , similar to (1).

There are two cases for the time to prepare the SF environment required by user u . If the SF that the user u relies on is already activated on the target ES, it is not necessary to reactivate a new one, so the SF environment preparation time can be ignored. Instead, the target ES need to activate a new SF. In this article, to simplify the processing, we only consider the case where the SF preparation time is constant for each application, and the SF environment can be activated and adjusted with a short control frame at the beginning of the state context information synchronization.

To sum up, the service migration delay from the source ES e to the target ES e' is given by

$$I_{u,e'}^{\text{mig}}(t) = I_{u,e'}^{\text{sus}}(t) + I_{u,e'}^{\text{res}}(t) + I_{u,e'}^{\text{syn}}(t). \quad (8)$$

b) Communication delay: When the service migration is complete, mobile user u can access the target ES e' . The transmission delay of the tasks offloading from user u to the target ES e' can be expressed as follows:

$$I_{u,e'}^{\text{comm}}(t) = \frac{D_{u,e}(t)}{r_{u,e'}(t)} \quad (9)$$

where $r_{u,e'}$ denotes the data communication rate between the user u and the target ES e' , similar to (1).

c) Computing delay: We use $f_{u,e'}$ to represent the computational resources allocated by the target ES e' to the mobile user u , then the computational delay of the target ES e' to process the tasks from the mobile user u can be written as follows:

$$I_{u,e'}^{\text{comp}}(t) = \frac{D_{u,e}(t)C_{u,e}(t)}{f_{u,e'}(t)}. \quad (10)$$

Thus, when a user moves from the ES e to the vicinity of the ES e' , if service migration is performed and the service is migrated from the source ES e to the target ES e' , the task processing delay can be expressed as follows:

$$I_{u,e'}^m(t) = I_{u,e'}^{\text{comm}}(t) + I_{u,e'}^{\text{comp}}(t) + I_{u,e'}^{\text{mig}}(t). \quad (11)$$

B. Problem Formulation

In this article, we define the target ES as the closest ES to the predicted user location. According to (4) and (11), if the user continues to access the source ES after the move, the task processing latency is $I_{u,e}^m(t)$. If the server is migrated to the target ES and the task is processed through the target ES, the

task processing latency is $I_{u,e'}^m(t)$. We need to decide whether users continue to access the source ES or perform a service migration and access the target ES based on the computing and communication resource allocation policy of the target ES. Therefore, we set $I_u(t)$ to be the total task processing delay of user u at time t , and we have

$$I_u(t) = w_{u,e,e'}^m(t)I_{u,e}^m(t) + w_{u,e,e'}^m(t)I_{u,e'}^m(t) \quad (12)$$

where $w_{u,e,e'}^m(t)$ and $w_{u,e,e'}^m(t)$ are the binary service migration decision variable, when $w_{u,e,e'}^m(t) = 1$ and $w_{u,e,e'}^m(t) = 0$, the service in the source ES e are not migrated to the target ES e' , otherwise, $w_{u,e,e'}^m(t) = 0$ and $w_{u,e,e'}^m(t) = 1$. We aim to find the optimal service migration policy and resource allocation scheme that minimizes the total task processing delay, and considering the target ES resource constraint, the optimization problem can be formulated as

$$\mathcal{P} : \underset{\substack{w_{u,e,e'}^m(t), w_{u,e,e'}^m(t) \\ f_{u,e'}(t), b_{u,e'}(t)}}{\text{minimize}} \quad \lim_{T \rightarrow \infty} \frac{1}{T} \sum_{t=0}^T \sum_{u \in \mathcal{U}} I_u(t) \quad (13a)$$

$$\text{s.t.} \quad \sum_{u \in \mathcal{U}} \alpha_{u,e'}(t) \cdot f_{u,e'}(t) \leq F_{u,e'}(t) \quad (13b)$$

$$\sum_{u \in \mathcal{U}} \alpha_{u,e'}(t) \cdot b_{u,e'}(t) \leq B_{u,e'}(t) \quad (13c)$$

$$w_{u,e,e'}^m(t) + w_{u,e,e'}^m(t) = 1 \quad (13d)$$

$$w_{u,e,e'}^m(t), w_{u,e,e'}^m(t) \in \{0, 1\} \quad (13e)$$

$$\sum_{e' \in \mathcal{E}} \alpha_{u,e'}(t) = 1 \quad (13f)$$

$$\alpha_{u,e'}(t) \in \{0, 1\} \quad (13g)$$

$$\forall u \in \mathcal{U} \quad \forall e' \in \mathcal{E} \quad (13h)$$

where $\alpha_{u,e'}(t)$ represents the link relationship between user u and target ES e' at time t , and we have

$$\alpha_{u,e'}(t) = \begin{cases} 1, & \text{if user } u \text{ is served by edge server } e' \\ 0, & \text{otherwise.} \end{cases} \quad (14)$$

Constraint (13b) denotes that the CPU requirements of all users connected to the ES e' cannot exceed the available CPU capacity of the ES e' , where $F_{u,e'}(t)$ denotes the available computing capacity of e' at time t . Constraint (13c) represents that the bandwidth allocated to all users connected to the ES e' for offloading cannot exceed the available bandwidth of the ES e' , where $B_{u,e'}(t)$ denotes the available bandwidth of e' at time t . In particular, after the user u -dependent services are migrated, the corresponding computational and bandwidth resources on the source ES need to be released, meanwhile, the corresponding resources on the target ES are occupied, so that $F_{u,e'}(t)$ and $B_{u,e'}(t)$ in constraints (13b) and (13c) are dynamically changing over time. Constraints (13d) and (13e) indicate that for migration decisions within each decision cycle t , there are just two options for nonmigration and migration. Constraints (13f) and (13g) indicate that each user can be served by only one ES in each decision cycle t .

Since the service migration decision variables $w_{u,e,e'}^m(t)$ and $w_{u,e,e'}^m(t)$ are binary variables, the problem $\mathcal{P}$ is nonconvex. In addition, we consider the user mobility, ES resource heterogeneity and limited, which are difficult to solve with traditional

optimization algorithms. In contrast, DRL can update the SMRA policies by interacting with the environment in real time, which makes it possible to adapt to the dynamic changes of service requirements in emerging IoT systems. Therefore, we present a DRL-based algorithm to solve the problem $\mathcal{P}$.

IV. DRL-BASED SMRA APPROACH

In this section, we first reformulate the delay minimization problem as MDP and define the state, action, and reward function. Then we propose a DRL-based SMRA algorithm.

A. DRL-Based Framework

The problem $\mathcal{P}$ is described as MDP which mainly consists of three parts: 1) state; 2) action; and 3) reward.

1) *State*: At the beginning of each time slot, the controller observes system states, which includes the predicted user location, the location of the ES closest to the predicted location, resources allocated to users by the source ES, and the available resources of the target ES. In particular, according to the problem $\mathcal{P}$, when the computational and bandwidth resources allocated to the user by the source ES e are known, the resources available at the source ES e do not have an impact on the total task processing latency. Thus, the computational resources and bandwidth of source ES e need not be considered in the state space of reinforcement learning. The system state at time t can be defined as follows:

$$s_t \triangleq \{L_u^{pre}(t), L_{u,e'}^{pre}(t), f_{u,e}(t), b_{u,e}(t), F_{u,e'}(t), B_{u,e'}(t) | u \in \mathcal{U}\} \quad (15)$$

where

- 1) $L_u^{pre}(t)$ represents the predicted location of the user at the next time t , where to determine the location of the target ES capable of service migration, we predict the user's location $L_u^{pre}(t)$ at time $t+1$ by the LSTM [18] algorithm, and the ES closest to the $L_u^{pre}(t)$ is the target ES for service migration. The prediction process is described in detail in Section IV-B.
- 2) $L_{u,e'}^{pre}(t)$: We use e' represents the target ES which is the nearest ES to $L_u^{pre}(t)$, and $L_{u,e'}^{pre}(t)$ is the location of the target ES e' . There are many ESs in the vicinity of the user, and we take the ES closest to the predicted user location as the target ES and add it to the current state as a known value. First, the size of the state and action space can be greatly reduced, avoiding the exponential computational complexity due to global search. Second, in general, if the user device has not yet established a connection with another BS, it will automatically connect to the nearest BS [10], [11], [12]. If the user device wants to access resources on other ESs, it needs to perform BS handover first. According to the X2 handover principle in 5G environments, the user device needs to continuously send measurement reports to the currently connected BS, which decides whether handover is required based on these measurement reports, and if handover is required, the BS will notify the core network to decide which target BS to handover to. However, too much switching increases the

complexity of scheduling between ESs, core network, BSs, and user devices, consumes a lot of energy, and increases latency [16], [17]. Therefore, considering the computational complexity and handoff overhead, this article defines the ES closest to the predicted user location as the target ES.

- 3) $f_{u,e}(t)$ denotes the computing resources allocated to users by ES e at time t .
- 4) $b_{u,e}(t)$ denotes the bandwidth resources allocated to users by ES e at time t .
- 5) $F_{u,e'}(t)$ denotes the remaining available computing resources of the target ES e' at time t .
- 6) $B_{u,e'}(t)$ denotes the remaining available bandwidth of the target ES e' at time t .

2) *Action*: In state s_t , the system controller needs to decide whether to perform service migration and determine computation and communication resources that the target ES allocates to users. Therefore, the action of mobile user u at time t can be defined as follows:

$$a_t \triangleq \{w_{u,e,e'}^{nm}(t), w_{u,e,e'}^m(t), f_{u,e'}(t), b_{u,e'}(t) | u \in \mathcal{U}\} \quad (16)$$

where

- 1) $w_{u,e,e'}^{nm}(t), w_{u,e,e'}^m(t)$ represents service migration decisions of user u at time t .
- 2) $f_{u,e'}(t)$ represents the computing resources allocated to user u by the target ES e' at time t .
- 3) $b_{u,e'}(t)$ represents the bandwidth resources allocated to user u by the target ES e' at time t .

3) *Reward*: In state s_t , the system executing the action a_t will obtain a reward value. For the problem $\mathcal{P}$, our optimization objective is to minimize the task processing delay. Therefore, the reward function can be expressed by

$$r_t = -\mathbb{E} \left[\sum_{u \in \mathcal{U}} I_u(t) \right]. \quad (17)$$

Since $I_u(t)$ is determined by weighting the task processing time $I_{u,e'}^m(t)$ in the migration case and the task processing time $I_{u,e}^{nm}(t)$ in the nonmigration case, as in (4), (11), and (12). If the target server has insufficient or even no available remaining computational and bandwidth resources, then it cannot allocate sufficient resources for users to perform task processing, resulting in a longer task processing time for the target ES than for the source ES, i.e., the reward value obtained from the migration action is lower than that obtained from the nonmigration action, and our migration decision variables $w_{u,e,e'}^{nm}(t)$ and $w_{u,e,e'}^m(t)$ are 0 or 1 decisions and our goal is to find a strategy that yields a high-reward value, so we would choose not to migrate the service and continue to access the source ES. Otherwise, we will choose to migrate the service.

In reinforcement learning, the policy is defined as a mapping from state to actions $\pi : \mathcal{S} \rightarrow \mathcal{A}$, where $\mathcal{S}$ represents the system states, and $\mathcal{A}$ represents the system actions. The objective is to find the optimal policy π^* that maximizes long-run expected rewards through interaction with the environment

$$\pi^* = \arg \max_{\pi} \mathbb{E} \left[\sum_t^T \gamma^t r(s_t, a_t) \right] \quad (18)$$

where T represents the episode termination time step and $\gamma \in [0, 1]$ is the discount factor. To solve π^* , the Q -learning algorithm [15] uses Q -values to represent state-action pairs, which are stored in a Q -matrix that is updated according to the rewards of each policy. However, the huge size of $\mathcal{S}$ and $\mathcal{A}$ will result in a huge size of Q -matrix. To the above issues, DRL combines deep neural network (DNN) and RL by inputting the state-action pairs into a DNN to output Q -values. The Bellman equation of action-value function (i.e., Q -function) is given by

$$Q(s_t, a_t) = \mathbb{E}_{r_t, s_{t+1}} \left[r_t + \gamma \max_{a_{t+1} \in \mathcal{A}} Q(s_{t+1}, a_{t+1}) \right]. \quad (19)$$

In the classical DRL algorithm, in order to address the shortcomings in solving the Q -function with DNN, the deep Q -network (DQN) algorithm uses “empirical replay” techniques and asynchronous updating of the Q -network. However, DQN is only applicable to handle discrete action spaces, for the continuous action space, DQN cannot efficiently calculate the optimal action $a^*(s) = \max_a Q^*(s, a)$, and it is difficult to obtain greedy strategies. To overcome this problem, the DDPG algorithm is proposed, which introduces a deterministic actor network to output approximate maximized Q -value and updates the actor network by gradient ascent. However, the action space shown in (16) is a complex discrete-continuous hybrid action space, and just using DQN or DDPG is not sufficient to solve this challenge. For this reason, we use PDQN to solve the hybrid action space challenge, which combines the features of DQN and DDPG.

B. SMRA Algorithm Based on LSTM and PDQN

In this article, we need to find an optimal SMRA policy based on the current state of the IoT system, including determining where to migrate, whether to migrate, and how to allocate resources. To determine the service migration location, we first use the LSTM [18] algorithm to predict the user's future location $L_u^{pre}(t)$ from historical user location data and add the ES closest to the user's next location as the target ES for service migration to the current state. Then, we use the PDQN algorithm to determine whether to perform service migration and how to allocate resources. DRL is a long-term iterative optimization process, while LSTM is suitable for handling time series data. We use the location predicted by LSTM as one of the states of DRL, allowing the intelligent system to learn both the location prediction model and the decision model. In addition, the SMRA algorithm based on LSTM and PDQN can be embedded directly into the intelligent system without relying on external mapping applications, making it easier to deploy and maintain. We hope that the SMRA algorithm will improve the intelligence of the system, allowing the intelligent system to make optimal SMRA plans before the user moves, ensuring service continuity and resource utilization.

The SMRA algorithm gives actions for SMRA based on the real-time state of the system and calculates the reward values after the actions are executed, after which the system enters a new state. Based on these empirical data of states, actions, action reward and next state values, the DRL model is trained and updated. Finally, the trained DRL model is used to implement intelligent management of the IoT system.

We separate the action space of (16) and use k to represent the discrete actions $\{w_{u,e,e'}^{nm}(t), w_{u,e,e'}^m(t)\}$ and x_k ($x_k \in \mathcal{X}_k$, $\mathcal{X}_k$ to represent the set of continuous actions) to represent the continuous actions $\{f_{u,e'}(t), b_{u,e'}(t)\}$. More specifically, we can denote the discrete-continuous hybrid action space $\mathcal{A}$ as

$$\mathcal{A} = \{(k, x_k) \mid x_k \in \mathcal{X}_k, \text{ for all } k \in K\} \quad (20)$$

where K is the set of discrete actions. We represent the action value function $Q(s, a)$ by $Q(s, k, x_k)$, in which $s \in \mathcal{S}$, $a \in \mathcal{A}$, $k \in K$, and $x_k \in \mathcal{X}_k$. We use k_t to represent the discrete action chosen at time t and x_{k_t} to represent the corresponding continuous parameter. Then the Bellman equation can be written by

$$Q(s_t, k_t, x_{k_t}) = \mathbb{E}_{r_t, s_{t+1}} \left[r_t + \gamma \max_{k \in K} \sup_{x_k \in \mathcal{X}_k} Q(s_{t+1}, k, x_k) \right]. \quad (21)$$

For the above Bellman equation, we need to compute $x_k^* = \operatorname{argsup}_{x_k \in \mathcal{X}_k} Q(s_{t+1}, k, x_k)$ for each $k \in K$, and choose the largest $Q(s_{t+1}, k, x_k^*)$. When function Q is fixed, $x_k^* = \operatorname{argsup}_{x_k \in \mathcal{X}_k} Q(s_{t+1}, k, x_k)$ can be regarded as a function $x_k^Q: \mathcal{S} \rightarrow \mathcal{X}_k$ for $\forall s \in \mathcal{S}$ and $\forall k \in K$ that maps the state space to the continuous domain of action parameters. Then the Bellman equation of (21) can be rewritten by

$$Q(s_t, k_t, x_{k_t}) = \mathbb{E}_{r_t, s_{t+1}} \left[r_t + \gamma \max_{k \in K} Q(s_{t+1}, k, x_k^Q(s_{t+1})) \right]. \quad (22)$$

Similar to DQN, we use DNN $Q(s, k, x_k; \omega)$ to approximate $Q(s, k, x_k)$, in which ω represents the network parameters. For such $Q(s, k, x_k; \omega)$, similar to DDPG, we approximate $x_k^Q(s)$ by a deterministic policy network $x_k(\cdot; \theta): \mathcal{S} \rightarrow \mathcal{X}_k$, in which θ represents the parameter of deterministic policy network. It means that when ω is fixed, the objective of PDQN is to find the corresponding parameter θ such that

$$Q(s, k, x_k(s; \theta); \omega) \approx \sup_{x_k \in \mathcal{X}_k} Q(s, k, x_k; \omega) \quad \forall k \in K. \quad (23)$$

Then we use the following least mean squares loss function for ω similar to DQN.

$$\ell_t^Q(\omega) = \frac{1}{2} [Q(s_t, k_t, x_{k_t}; \omega) - y_t]^2 \quad (24)$$

where

$$y_t = \begin{cases} r_t, & \text{if } s_{t+1} \text{ is the terminal state} \\ r_t + \max_{k \in K} \gamma Q(s_{t+1}, k, x_t(s_{t+1}; \theta_t); w_t), & \text{else} \end{cases} \quad (25)$$

is evaluated by the target network and s_{t+1} denotes the next state following the adoption of the hybrid action (k, x_k) . Furthermore, as our objective is to find θ which maximize $Q(s, k, x_k(s; \theta); \omega)$ while ω is fixed, we perform the following loss function for θ :

$$\ell_t^\Theta(\theta) = - \sum_{k=1}^K Q(s_t, k, x_t(s_t; \theta); w_t). \quad (26)$$

In addition, the value network and deterministic policy network weights w_t and θ_t are updates via gradient descent according to

$$\omega_{t+1} \leftarrow \omega_t - \alpha_t \nabla_\omega \ell_t^Q(\omega_t) \quad (27)$$

$$\theta_{t+1} \leftarrow \theta_t - \beta_t \nabla_\theta \ell_t^\Theta(\theta_t) \quad (28)$$

where α_t and β_t are the learning rate.

TABLE II
SIMULATION PARAMETERS

Parameter	Values
Available channel bandwidth, B	10MHz [28]
Transmission powers of user u , $p_{u,e}$	24dBm [28]
Gaussian noise power, σ	-100dBm/Hz [26]
Available CPU capacity of edge server e' , $F_{e'}$	150×10^8 cycles/s [28]
The task data size, $D_{u,e}$	[200, 300]KB [4]
The number of CPU cycles required to compute one-bit task data, $C_{u,e}$	[300, 500]cycles/bit [4]
Computation resource allocated to user u by edge server e , $f_{u,e}$	$[f_{u,e}^{min}, f_{u,e}^{max}]$
Minimum value of computation resource allocated to user by edge server, $f_{u,e}^{min}$	10×10^7 cycles/s
Maximum value of computation resource allocated to user by edge server, $f_{u,e}^{max}$	80×10^7 cycles/s
Bandwidth resource allocated to user u by edge server e , $b_{u,e}$	$[b_{u,e}^{min}, b_{u,e}^{max}]$
Minimum value of bandwidth resource allocated to user by edge server, $b_{u,e}^{min}$	0.3 MHz
Maximum value of bandwidth resource allocated to user by edge server, $b_{u,e}^{max}$	5 MHz
The size of state context data of user u , $D_{u,e}^{mig}$	[1, 8]MByte [4]
The proportion of computational resources allocated to the checkpoint function, $\beta_{u,e'}$	[20, 30]% [4]
Processing intensity required to suspend SI, $C_{u,e'}^{sus}$	[228, 538]cycles/bit [4]
Processing intensity required to resume SI, $C_{u,e'}^{res}$	[350, 450]cycles/bit [4]
Computing capacity of SF required by user, $f_{u,e'}^{SF}$	$[200, 300] \times 10^6$ Hz [4]
Discount Factor, γ	0.9 [26]
Replay size, $\mathcal{R}$	1000 [26]
Mini-batch size M	25 [26]
The greedy policy parameter, ϵ	0.05
Averaging rate, v	0.01 [27]

V. SIMULATION RESULTS AND DISCUSSION

A. Experimental Settings

We validate our SMRA scheme using a real-world data set containing the GPS trajectories of 10 357 cabs in Beijing [24]. The trajectory data of each cab includes longitude, latitude and time, and its longitude and latitude change dynamically with time. The data set was released by Microsoft Research Asia, and the release date was from 2 February 2008 to 8 February 2008. In our experiments, we treat each mobile cab in the data set as a mobile user.

In the default case, we randomly select the trajectory data of 70 mobile users in a certain area from the data set and deploy 60 ESs, setting the distance between the ESs to 5 km to ensure that the deployed ESs can cover the selected 70 users. The channel gain is set to $(d_{u,e})^{-\alpha}$, where $d_{u,e}$ represents the distance between user u and ES e and $\alpha = 3$ is the pathloss factor [25]. We use Euclidean distance formula to calculate the distance. The default experiment parameters settings are as Table II. With the experimental parameters taking default values, we set the execution period of the cloud server (i.e., SMRA execution cycle) to every 10 min.

To verify the superiority of the SMRA algorithm presented in this article, we compare it by the following five methods.

- 1) *DDPG-Based Scheme*: We select the state-of-the-art method from literature [10] for our comparison experiments. This literature used a DDPG-based algorithm

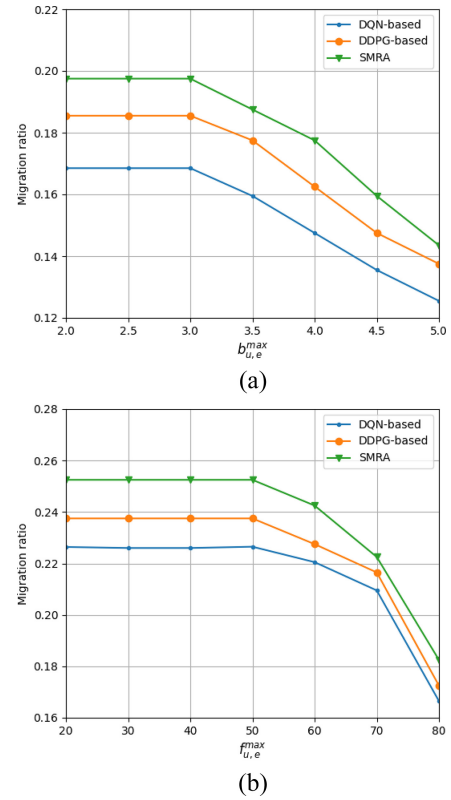


Fig. 3. Impact of different maximum user resource requirement ranges on service migration ratio. (a) Maximum value of bandwidth resource allocated to user by ES. (b) Maximum value of computation resource allocated to user by ES.

to solve the computational offloading and resource allocation problem. In order to adapt DDPG to a hybrid action space with discrete and continuous actions, the literature uses a rounding technique to refine the binary offloading decision.

- 2) *DQN-Based Scheme*: We discretize the continuous resource allocation values in the action space of this article by approximating each x_t with a discrete subset, and then solve them using the DQN-based algorithm.
- 3) *Random Migrate Scheme (RMS)*: When the ES closest to the user at time t and $t + 1$ is the same ES, we do not perform migration, otherwise, the service randomly chooses whether to migrated to the ES closest to the user.
- 4) *Always Migrate Scheme (AMS)*: This migration scheme migrates the services required by the user to the ES closest to the user as the user moves.
- 5) *Never Migrate Scheme (NMS)*: The service will not be migrated no matter how the user moves.

B. Parameter Analysis

In practice, different users have different resource requirements, so we set the computing resources and bandwidth allocated to users by the ES varying within a certain range, and we adjust the size of the maximum value of the range of computing resources and bandwidth and conduct two sets of experiments to compare the proportion of users migrating and the average task processing latency, respectively. Fig. 3 shows

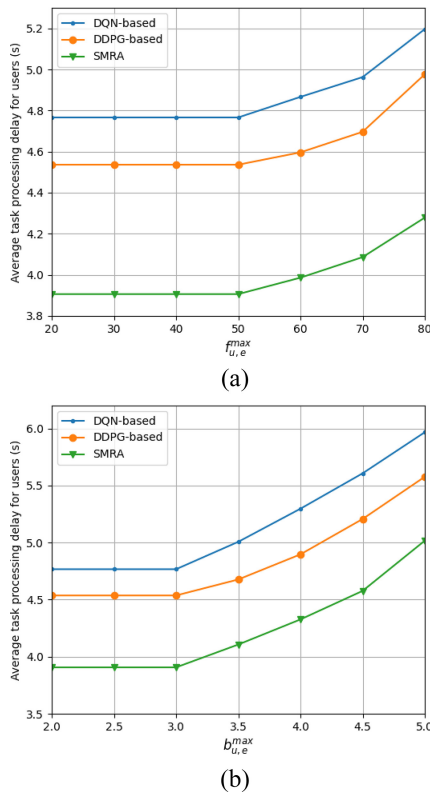


Fig. 4. Impact of different maximum user resource requirement ranges on average user task processing delay. (a) Maximum value of bandwidth resource allocated to user by ES. (b) Maximum value of computation resource allocated to user by ES.

the variation of the proportion of user migration for different algorithms when the upper limits of the random range of computational resources and bandwidth allocated to users are increased. We can see from the figure that when the allocated computational resources and bandwidth to users are relatively small, the resource demand of each user can be satisfied, so the migration rate remains constant as the resource demand of users increases. When the resources allocated to users reach a certain value, the ES will not be able to satisfy the resource demand of each user, which will result in fewer users that can be served by the ES and a lower migration ratio.

Fig. 4 shows the variation in the average task processing delay of users when the upper limit of the random range of allocated computing resources and bandwidth increases. We can see from the figure that when the allocated computing resources and bandwidth to users are relatively small, the resource demand of each user can be satisfied and the average task processing latency of each user remains constant. When the resources allocated to users reach a certain value, the ES will not be able to satisfy the resource demand for each user, This will result in fewer users to be served by ESs and lower migration rates.

Fig. 5 illustrates the variation of the average task processing delay of users as the number of iterations increases for discount factors of 0.1, 0.2, 0.5, and 0.9, respectively, from which it can be seen that the average delay decreases gradually with the number of iterations for different discount factors and finally converges to an optimal solution.

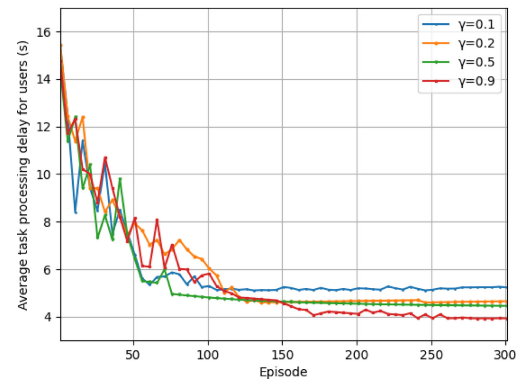


Fig. 5. Impact of discount factors on convergence.

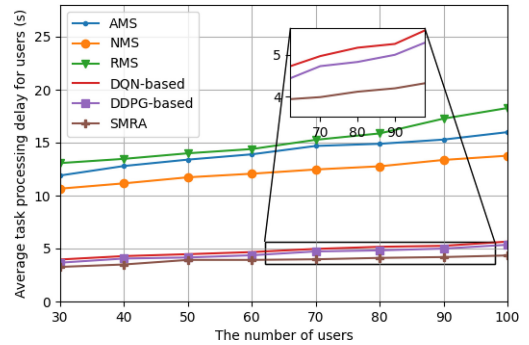


Fig. 6. Impact of the number of users on average task processing delay.

C. Comparison Experiments

Fig. 6 compares the performance of the six algorithms by adjusting the number of users. From the figure, we can see that as the number of users increases, the average task processing latency also increases, but the DRL-based scheme grows relatively slowly, and using the DRL-based scheme to address the SMRA issues is significantly better than the other three schemes. More specifically, compared with always migration scheme, the DRL-based scheme can choose the appropriate policy for service migration according to the current system state, thus avoiding frequent service migration, and compared with never migration scheme, the DRL-based scheme avoids excessive latency between ESs and users due to long communication distance. In addition, among the three DRL-based schemes, the SMRA scheme proposed in this article can achieve lower average task processing latency compared to DQN-based and DDPG-based schemes due to better preservation of the action space structure.

Fig. 7 illustrates the performance comparison of six schemes as the number of ESs increases. We can see from the figure as the number of ESs increases, the resources of ESs also increase and therefore the average task processing latency becomes smaller. The DRL-based methods have a smaller average task processing latency compared to the other three solutions because of the tradeoff between migration latency and nonmigration latency. The SMRA scheme proposed in this article is consistently optimal in different performance comparisons.

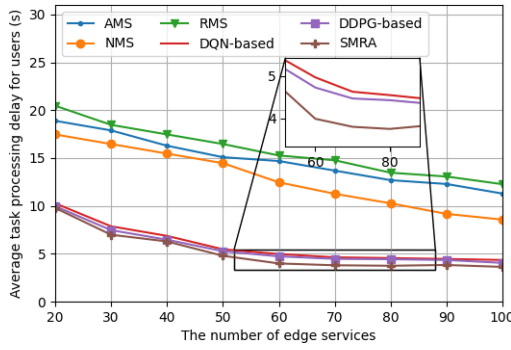


Fig. 7. Impact of the number of ESs on average task processing delay.

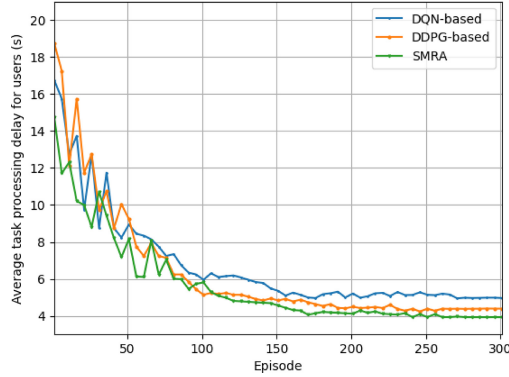


Fig. 8. Iterative changes of different schemes.

In order to analyze the superiority of the SMRA algorithm from multiple perspectives, we further compare the SMRA algorithm with the DQN-based scheme and the DDPG-based scheme. Fig. 8 presents the variation of user average task processing latency with increasing number of iterations for different DRL-based schemes, from which we can find the average task processing delay of all three schemes gradually decreases as the number of iterations increases and finally converges to an optimal solution. However, the SMRA scheme can guarantee the minimum average user task processing delay for the final convergence.

VI. CONCLUSION

This article investigated the joint optimization problem of SMRA in edge IoT systems, fully considering the different resource requirements and mobility of IoT users, as well as the limited resources and heterogeneity constraints of ESs, with the aim of minimizing the access delay of IoT users. This article formulated the joint optimization SMRA problem as a MDP and proposed a DRL-based joint SMRA scheme, whereby the LSTM algorithm and PDQN algorithm can sense the user mobility and decide whether to migrate a service, where to migrate it, and how to reallocate the resources, effectively ensuring the resource utilization and service continuity of the IoT system and reducing the access delay of IoT users. In addition, the PDQN algorithm effectively solves the complex discrete-continuous hybrid action space problem in the SMRA problem. Finally, we conducted simulation experiments using real data sets of Beijing cab trajectories to

verify the effectiveness of the SMRA algorithm and conducted comparison experiments to prove the superiority of the SMRA algorithm.

REFERENCES

- [1] T. Taleb, A. Ksentini, and P. A. Frangoudis, "Follow-Me cloud: When cloud services follow mobile users," *IEEE Trans. Cloud Comput.*, vol. 7, no. 2, pp. 369–382, Apr.–Jun. 2019.
- [2] T. Ouyang, Z. Zhou, and X. Chen, "Follow me at the edge: Mobility-aware dynamic service placement for mobile edge computing," *IEEE J. Sel. Areas Commun.*, vol. 36, no. 10, pp. 2333–2345, Oct. 2018.
- [3] B. E. Mada, M. Bagaa, T. Taleb, and H. Flinck, "Latency-aware service placement and live migrations in 5G and beyond mobile systems," in *Proc. IEEE ICC'20*, Dublin, Ireland, 2020, pp. 1–6.
- [4] Y. Chen, Y. Sun, C. Wang, and T. Taleb, "Dynamic task allocation and service migration in edge-cloud IoT system based on deep reinforcement learning," *IEEE Internet Things J.*, vol. 9, no. 18, pp. 16742–16757, Sep. 2022.
- [5] S. Wang, R. Urgaonkar, M. Zafer, T. He, K. Chan, and K. K. Leung, "Dynamic service migration in mobile edge computing based on Markov decision process," *IEEE/ACM Trans. Netw.*, vol. 27, no. 3, pp. 1272–1288, Jun. 2019.
- [6] R. A. Addad, D. L. C. Dutra, T. Taleb, and H. Flinck, "Toward using reinforcement learning for trigger selection in network slice mobility," *IEEE J. Sel. Areas Commun.*, vol. 39, no. 7, pp. 2241–2253, Jul. 2021.
- [7] R. A. Addad, D. L. C. Dutra, T. Taleb, and H. Flinck, "AI-based network-aware service function chain migration in 5G and beyond networks," *IEEE Trans. Netw. Service Manag.*, vol. 19, no. 1, pp. 472–484, Mar. 2022.
- [8] Z. Liang, Y. Liu, T.-M. Lok, and K. Huang, "Multi-cell mobile edge computing: Joint service migration and resource allocation," *IEEE Trans. Wireless Commun.*, vol. 20, no. 9, pp. 5898–5912, Sep. 2021.
- [9] Y. Liu, H. Yu, S. Xie, and Y. Zhang, "Deep reinforcement learning for offloading and resource allocation in vehicle edge computing and networks," *IEEE Trans. Veh. Technol.*, vol. 68, no. 11, pp. 11158–11168, Nov. 2019.
- [10] Y. Dai, K. Zhang, S. Maharjan, and Y. Zhang, "Edge intelligence for energy-efficient computation offloading and resource allocation in 5G beyond," *IEEE Trans. Veh. Technol.*, vol. 69, no. 10, pp. 12175–12186, Oct. 2020.
- [11] "Evolved universal terrestrial radio access (E-UTRA); radio resource control (RRC); protocol specification; (Release 8)," 3GPP, Sophia Antipolis, France, Rep. TS 36.331 V8.20.0, Jun. 2013.
- [12] J. Liu, S. Zhang, H. Nishiyama, N. Kato, and J. Guo, "A stochastic geometry analysis of D2D overlaying multi-channel downlink cellular networks," in *Proc. IEEE Conf. Comput. Commun. (INFOCOM)*, 2015, pp. 46–54.
- [13] C. Wang, F. R. Yu, C. Liang, Q. Chen, and L. Tang, "Joint computation offloading and interference management in wireless cellular networks with mobile edge computing," *IEEE Trans. Veh. Technol.*, vol. 66, no. 8, pp. 7432–7445, Aug. 2017.
- [14] M. Horii, Y. Kojima, and K. Fukuda, "Stateful process migration for edge computing applications," in *Proc. IEEE Wireless Commun. Netw. Conf. (WCNC)*, Barcelona, Spain, 2018, pp. 1–6.
- [15] C. J. C. H. Watkins and P. Dayan, "Q-learning," *Mach. Learn.*, vol. 8, nos. 3–4, pp. 279–292, 1992.
- [16] C.-P. Lee and P. Lin, "Modeling delay timer algorithm for handover reduction in heterogeneous radio access networks," *IEEE Trans. Wireless Commun.*, vol. 16, no. 2, pp. 1144–1156, Feb. 2017.
- [17] X. Wang et al., "Handover reduction in virtualized cloud radio access networks using TWDM-PON fronthaul," *J. Opt. Commun. Netw.*, vol. 8, no. 12, pp. B124–B134, Dec. 2016.
- [18] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Comput.*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [19] H. Wang, H. He, Y. Bai, and H. Yue, "Parameterized deep Q-network based energy management with balanced energy economy and battery life for hybrid electric vehicles," *Appl. Energy*, vol. 320, Aug. 2022, Art. no. 119270.
- [20] V. Mnih et al., "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, 2015.
- [21] C. Qiu, Y. Hu, Y. Chen, and B. Zeng, "Deep deterministic policy gradient (DDPG)-based energy harvesting wireless communications," *IEEE Internet Things J.*, vol. 6, no. 5, pp. 8577–8588, Oct. 2019.

- [22] Q. Tang et al., "Distributed task scheduling in serverless edge computing networks for the Internet of Things: A learning approach," *IEEE Internet Things J.*, vol. 9, no. 20, pp. 19634–19648, Oct. 2022.
- [23] Y. Liu, H. Wang, M. Peng, J. Guan, and Y. Wang, "An incentive mechanism for privacy-preserving crowdsensing via deep reinforcement learning," *IEEE Internet Things J.*, vol. 8, no. 10, pp. 8616–8631, May 2021.
- [24] J. Yuan, Y. Zheng, X. Xie, and G. Sun, "Driving with knowledge from the physical world," in *Proc. 17th ACM SIGKDD Int. Conf. Knowl. Discov. Data Min.*, pp. 316–324, 2011.
- [25] X. Chen, L. Jiao, W. Li, and X. Fu, "Efficient multi-user computation offloading for mobile-edge cloud computing," *IEEE/ACM Trans. Netw.*, vol. 24, no. 5, pp. 2795–2808, Oct. 2016.
- [26] Q. Yuan, J. Li, H. Zhou, T. Lin, G. Luo, and X. Shen, "A joint service migration and mobility optimization approach for vehicular edge computing," *IEEE Trans. Veh. Technol.*, vol. 69, no. 8, pp. 9041–9052, Aug. 2020.
- [27] M. Dorokhova, Y. Martinson, C. Ballif, and N. Wyrsh, "Deep reinforcement learning control of electric vehicle charging in the presence of photovoltaic generation," *Appl. Energy*, vol. 301, Nov. 2021, Art. no. 117504.
- [28] J. Ren, G. Yu, Y. He, and G. Y. Li, "Collaborative cloud and edge computing for latency minimization," *IEEE Trans. Veh. Technol.*, vol. 68, no. 5, pp. 5031–5044, May 2019.



Fangzheng Liu received the M.S. degree in computer science and technology from North China University of Technology, Beijing, China, in 2017. She is currently pursuing the Ph.D. degree with the School of Information Science and Engineering, China University of Petroleum, Beijing.

From July 2022 to July 2023, she was a visiting Ph.D. student with the Center for Wireless Communications, University of Oulu, Oulu, Finland. Her current research interests include

edge/cloud/services computing, performance modeling, and optimization.



Hao Yu (Member, IEEE) received the B.S. and Ph.D. degrees in communication engineering from Beijing University of Posts and Telecommunications, Beijing, China, in 2015 and 2020, respectively.

He was also a joint-supervised Ph.D. student with Politecnico di Milano, Milan, Italy. He is currently a Postdoctoral Researcher with the Center of Wireless Communications, Oulu University, Oulu, Finland. His research interests include intelligent edge network, time sensitive networks, and 6G deterministic networking.



Jiwei Huang (Senior Member, IEEE) received the B.Eng. and Ph.D. degrees in computer science and technology from Tsinghua University, Beijing, China, in 2009 and 2014, respectively.

He was a Visiting Scholar with Georgia Institute of Technology, Atlanta, GA, USA. He is currently a Professor and the Vice Dean of the College of Information Science and Engineering/College of Artificial Intelligence, China University of Petroleum, Beijing, and the Director of Beijing Key Laboratory of Petroleum Data Mining. He has

authored or coauthored one book and more than 60 articles in international journals and conference proceedings, including IEEE TRANSACTIONS ON MOBILE COMPUTING, IEEE TRANSACTIONS ON SERVICES COMPUTING, IEEE TRANSACTIONS ON CLOUD COMPUTING, IEEE TRANSACTIONS ON VEHICULAR TECHNOLOGY, IEEE INTERNET OF THINGS JOURNAL, ACM SIGMETRICS, and IEEE ICWS. His research interests include Internet of Things, edge computing, and services computing.

Dr. Huang is currently on the editorial board of *Chinese Journal of Electronics and Scientific Programming*, and served as a TPC member for IEEE ICWS, CollaborateCom, and PRICAI. He is a Senior Member of CCF.



Tarik Taleb (Senior Member, IEEE) received the B.E. degree (with Distinction) in information engineering and the M.Sc. and Ph.D. degrees in information sciences from Tohoku University, Sendai, Japan, in 2001, 2003, and 2005, respectively.

He is currently a Professor with the Center of Wireless Communications, University of Oulu, Oulu, Finland. He is the Founder and the Director of the MOSAIC Lab, Oulu. From October 2014 and December 2021, he was a Professor with the School

of Electrical Engineering, Aalto University, Espoo, Finland. Prior to that, he was a Senior Researcher and the 3GPP Standards Expert with NEC Europe Ltd., Heidelberg, Germany. Before joining NEC and till March 2009, he was an Assistant Professor with the Graduate School of Information Sciences, Tohoku University, in a lab fully funded by KDDI, the second largest mobile operator in Japan. From October 2005 to March 2006, he was a Research Fellow with the Intelligent Cosmos Research Institute, Sendai. His research interests lie in the field of telco cloud, network softwarization and network slicing, AI-based software-defined security, immersive communications, mobile multimedia streaming, and next-generation mobile networking.

Dr. Taleb served as the General Chair for the 2019 Edition of the IEEE Wireless Communications and Networking Conference held in Marrakech, Morocco. He was the Guest Editor-in-Chief of the IEEE JOURNAL ON SELECTED AREAS IN COMMUNICATIONS Series on Network Softwarization and Enablers. He was on the editorial board of the IEEE TRANSACTIONS ON WIRELESS COMMUNICATIONS, *IEEE Wireless Communications Magazine*, IEEE INTERNET OF THINGS JOURNAL, IEEE TRANSACTIONS ON VEHICULAR TECHNOLOGY, IEEE COMMUNICATIONS SURVEYS AND TUTORIALS, and a number of Wiley journals. Till December 2016, he served as the Chair of the Wireless Communications Technical Committee. He has been also directly engaged in the development and standardization of the Evolved Packet System as a member of 3GPP's System Architecture Working Group 2.